

SoulLike

서버 권위 액션 구조 현대화 기술문서

레거시 액션 RPG의 **Server RPC** → **Reliable Multicast** 중심 전투 흐름을, 소유 클라이언트 예측 재생과 서버 확정 액션 상태 복제 구조로 단계적으로 현대화한 개인 프로젝트. 그래픽 완성도가 아니라 **멀티플레이 전투 동기화 · 액션 상태 복제 · 검증 가능한 리팩터링**을 설명합니다.

Server Authority

Action Replication

Client Prediction

GameplayTag · DataTable

Boss Skill Sync

Death / Respawn

AUTHOR

유재우

Unreal Gameplay Programmer

SoulLike · 개인 프로젝트 (레거시 현대화)

CORE WORK

- 액션 실행 단일 진입점 UActionComponent::TryAction 설계
- 소유 클라이언트 예측 재생 + 서버 확정 액션 상태 복제
- 장착 아이템 · 현재 액션 태그(GameplayTag) 최소 복제
- 피격 리액션 · Grux 보스 메테오 서버 권위 처리
- 사망 / 부활 — 서버 상태 복구와 클라이언트 표현 복구 분리

Project Overview

프로젝트 한 줄 소개, 목표와 제약을 정리합니다.

프로젝트 개요

Unreal Engine 5.7 기반 소울라이크 액션 RPG에서, 기존 전투 흐름을 **서버 권위 액션 상태 복제 구조**로 단계적으로 현대화했습니다. 평타 · 피격 · 장착 아이템 · 보스 스킬 · 사망/부활처럼 결과가 중요한 흐름에서 **서버와 클라이언트 결과가 어긋나는 문제**를 줄이는 것이 목표입니다.

Project	SoulLike — 서버 권위 액션 구조 현대화
Genre	소울라이크 액션 RPG (멀티플레이)
Engine	Unreal Engine 5.7 · C++
Networking	Unreal Replication · RPC · NetMulticast
Gameplay	GameplayTag · DataTable · Anim Notify
Validation	UnrealBuildTool · 2인 PIE · 별도 프로세스 NULL
Type	레거시 프로젝트 현대화 · 1인 개발

목표

- 플레이어와 AI 액션이 **서버 기준**으로 결정되고, 클라이언트에는 복제 상태/명시적 네트워크 이벤트로 표현되게 만든다.
- 전투 핵심 흐름에서 서버·클라이언트 결과가 어긋나는 문제를 줄인다.
- 한 번에 갈아엎지 않고, **빌드 가능한 작은 단위**로 수정하고 문서화한다.

제약

- 레거시 Unreal 프로젝트 현대화 — 대규모 재작성 대신 단계적 수정이 필요했다.
- AItem · Anim Notify · Character · Controller가 서로 직접 호출하는 구조가 남아 한 번에 교체하기 어려웠다.
- 게임플레이 복제 문제와 OSS 문제를 분리하기 위해 NULL/직접 접속 및 PIE 검증을 우선했다.

Implementation Scope

직접 구현한 범위, 외부 자산, AI 활용 경계를 구분합니다.

직접 설계 및 구현

- 액션 실행 단일 진입점 TryAction(FGameplayTag) 추가
- 소유 클라이언트 예측 재생 + 서버 확정 액션 상태 복제 설계
- FActiveAction{ActionTag, Sequence} 기반 반복 복제 트리거
- StateComponent::Type 서버 권위 갱신 · OnRep 전환
- 장착 아이템 Actor · 선택 아이템 참조 복제 정리
- AItem::CurrentActionTag 복제로 동일 DataTable 액션 재구성
- 피격 리액션을 서버 판정 후 Multicast 이벤트로 전파
- Grux 보스 메테오 / 공중 콤보 서버 권위 처리 · 널 가드 보강
- 사망/부활에서 서버 상태 복구와 클라이언트 표현 복구 분리
- 별도 프로세스 NULL 리슨서버/클라이언트 테스트 스크립트 작성
- 작업 로그 · 아키텍처 문서 · 검증 기록 작성

외부 리소스 · 기존 자산

캐릭터, 애니메이션, Niagara 이펙트, 일부 맵/에셋은 기존 프로젝트 리소스와 외부 에셋을 사용했습니다.
포트폴리오 평가 대상은 에셋 제작이 아니라 전투 액션 구조, 네트워크 동기화, 디버깅 및 리팩터링 과정입니다.

AI 활용 범위

- **Codex** — 코드 검색, 영향 범위 추적, 작업 로그 정리, 테스트 기준 문서화
- **UE-MCP** — 에디터 상태 확인, PIE 실행, 런타임 Pawn/Component 상태 확인 보조

직접 수행: 최종 구조 선택, 코드 수정 방향, 빌드 실패 원인 판단, 검증 기준 결정.

Tech Stack

전투 동기화와 리팩터링에 사용한 엔진·네트워크·툴링을 정리합니다.

사용 기술

구분	기술	사용 목적
Engine	Unreal Engine 5.7	C++ 기반 액션 RPG · PIE/별도 프로세스 검증
Language	C++	캐릭터 · 액션 · 아이템 · AI · 투사체 · 네트워크 로직 구현
Networking	Replication · RPC · NetMulticast	액션 상태 · 장착 아이템 · 피격/이펙트 이벤트 동기화
Gameplay	GameplayTag · DataTable · Anim Notify	액션 식별 · 데이터 조회 · 몽타주 타이밍 이벤트
AI	Behavior Tree · AIController · Blackboard	타겟 탐색 · 공격 의사결정 · 보스 스킬 실행
Online	OnlineSubsystem NULL / Steam 준비	로컬 직접 접속 테스트 · Steam 세션 전환 준비
Tooling	UnrealBuildTool · UE-MCP · Git	빌드 검증 · PIE 확인 · 변경 단위 기록

설계 키워드

- Server Authority
- Owner Prediction
- OnRep
- COND_SkipOwner
- Minimal Replication
- GameplayTag
- NetMulticast Event
- Null Guard

핵심 트레이드오프

전체 상태가 아니라 의미만 복제 — FActionData 전체 대신 액션의 의미를 나타내는 GameplayTag만 복제하고, 클라이언트가 동일 DataTable에서 액션 데이터를 재구성합니다. 네트워크 상태를 작게 유지하면서 서버·클라이언트가 같은 액션을 가리키도록 보장합니다.

Legacy Structure — Core Problem

액션마다 네트워크 패턴이 달랐던 기존 전투 흐름의 한계입니다.

기존 전투 흐름

액션마다 네트워크 패턴이 달랐던 초기 구조

- **Input OnMouseL / OnMouseR / OnQ / OnEvade**
ABasePlayer 입력 진입
- **Server RPC**
서버 요청
- **Reliable NetMulticast**
전체 표현 전파
- **EquipComponent::GetCurrentItem()**
현재 아이템 조회
- **AItem::OnDefaultAction / OnSkillAction**
아이템 액션 실행
- **AnimMontage 재생**
몽타주 표현
- **Anim Notify — 콤보·충돌·이동·종료**
Notify가 게임플레이 직접 호출

이 구조에서 발생할 수 있던 문제

- **입력감 저하** — 소유 클라이언트도 서버 왕복 후에야 평타 몽타주를 보게 되어 지연 환경에서 반응이 늦다.
- **상태 충돌** — 클라이언트가 StateComponent::Type 같은 복제 변수를 로컬에서 직접 변경할 수 있어 서버 상태와 충돌한다.
- **데이터 불일치** — SelectItem·CurrentData가 서버/클라이언트에서 같은 액션을 가리킨다는 보장이 부족하다.
- **AI 재현 불가** — AIController는 서버에만 존재하므로 결정한 애니메이션 속도·공격 타이밍을 클라이언트가 그대로 재현하기 어렵다.
- **책임 경계 모호** — Anim Notify가 판정·이동·콤보·종료까지 직접 호출해 책임 경계가 흐렸다.

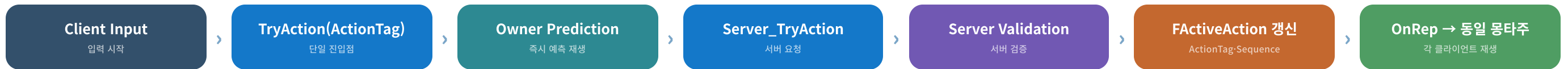
05

SOULLIKE · 서버 권위 액션

Target Architecture

입력은 클라이언트에서, 액션 상태 확정은 서버에서. 목표 흐름과 4원칙입니다.

목표 전투 액션 흐름



핵심 원칙 4가지

01 · 서버가 확정

입력은 클라이언트에서 시작할 수 있지만, **최종 액션 상태는 서버가 확정한다.**

02 · 태그 기준 재구성

클라이언트는 서버가 복제한 **액션 태그를 기준으로** 같은 DataTable 액션 데이터를 재구성한다.

03 · 예측 + SKIP OWNER

소유 클라이언트는 즉시 예측 재생하되, 서버 확정 상태와 충돌하지 않도록 **owner 복제는 건너뛴다.**

04 · 결과는 서버 권위

데미지 · 사망 · 부활 · 보스 스킬 판정처럼 **결과가 중요한 처리는 서버 권위로** 제한한다.

Core · Unified Action Component

액션 요청을 하나의 진입점으로 모으고, 예측 재생과 서버 확정을 분리합니다.

7-1 통일 액션 컴포넌트

UActionComponent는 평타 경로를 먼저 이전하기 위한 **최소 액션 레이어**입니다. 액션 요청을 하나의 진입점으로 모으고, 소유 클라이언트 예측과 서버 확정을 분리합니다.

- TryAction(FGameplayTag) — 액션 요청 단일 진입점
- 서버는 CommitAction()으로 검증과 실행을 처리
- 소유 클라이언트는 CanStartAction() 통과 시 즉시 PlayActionLocal() 실행 → 입력 지연 감소
- 서버 확정 액션은 ActiveAction으로 복제
- COND_SkipOwner** 복제로 예측 재생한 소유 클라이언트의 중복 재생 회피
- Sequence 증가 → 같은 ActionTag 반복에도 OnRep 재실행 보장

CODE PATH

Source/SoulLike/Component/ActionComponent.{h,cpp}
Source/SoulLike/Actor/Character/Player/BasePlayer.cpp
Source/SoulLike/Actor/Character/BaseCharacter.cpp

예측 · 확정 분기



소유 클라이언트는 예측 경로로 즉시 반응하고, 그 외 클라이언트는 복제된 ActiveAction으로 동일 몽타주를 재생합니다.

ACTIONCOMPONENT · 의사 코드

```
// 단일 진입점
void TryAction(FGameplayTag Tag){
    // 소유 클라이언트: 즉시 예측 재생
    if (IsOwnerLocal() && CanStartAction(Tag))
        PlayActionLocal(Tag);
    Server_TryAction(Tag);    // 서버에 요청
}

// 서버 권위 확정
void Server_TryAction_Impl(FGameplayTag Tag){
    if (!CommitAction(Tag)) return; // 검증 실패 → 무시
    ActiveAction.ActionTag = Tag;
    ActiveAction.Sequence++;    // 반복 트리거
}                                // COND_SkipOwner

void OnRep_ActiveAction(){ PlayActionLocal(ActiveAction.ActionTag); }
```

Core · State & Item Replication

상태 변경 권한을 서버로 모으고, 액션의 의미만 태그로 복제합니다.

7-2 상태 복제 기준 정리

UStateComponent::Type은 복제 변수였지만 클라이언트 로컬에서 직접 변경될 여지가 있었습니다. **상태 변경 권한을 서버로 모으는 것**이 목적입니다.

- Type을 ReplicatedUsing=OnRep_Type 기준으로 정리
- ChangeType()은 **서버 권위에서만** 상태 변경
- 액션 종료 시 서버는 Idle 복귀, 클라이언트 예측 잠금은 **몽타주 종료 기준**으로 해제하도록 분리

클라이언트가 임의로 액션 상태를 앞서 변경해 **다음 서버 복제와 충돌**하는 상황을 줄입니다.

7-3 장착 아이템 · 현재 액션 태그 복제

기존 Notify/콤보 구조는 GetCurrentItem()·CurrentData에 크게 의존 — 서버/클라이언트가 다른 데이터를 보면 같은 몽타주라도 판정·콤보가 어긋났습니다.

- 기본 아이템은 **서버에서만 스폰**
- SelectItem → ReplicatedUsing=OnRep_SelectItem 전환
- AItem을 복제 Actor로 전환 · bAlwaysRelevant·NetUpdateFrequency 조정
- CurrentActionTag 복제 → OnRep에서 로컬 DataTable 재조회로 CurrentData 재구성

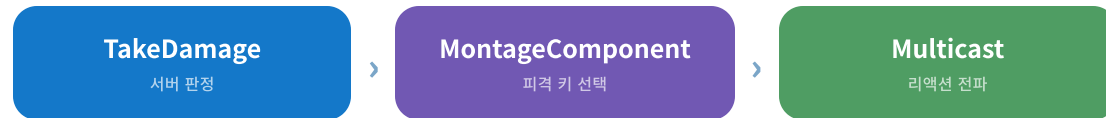
최소 복제 선택: FActionData 전체가 아니라 의미를 나타내는 **GameplayTag만 복제**해 네트워크 상태를 작게 유지합니다.

Core · Hit Reaction & Boss Sync

서버 판정 후 Multicast 이벤트로 표현을 전파하고, 보스 스킬을 서버 권위로 제한합니다.

7-4 피격 리액션 전파

서버에서 데미지를 처리해도 클라이언트에 피격 몽타주 재생 호출이 전달되지 않을 수 있었습니다.



- TakeDamage() — 서버에서 데미지/사망 판단
- 피격 몽타주 키 선택을 UMontageComponent로 이동
- MulticastPlayHitReaction()으로 각 클라이언트에 동일 리액션 전파

데미지 판정은 서버, 리액션 재생은 모든 클라이언트가 동일하게 수행하는 구조.

7-5 Grux 메테오 · 보스 스킬 동기화

보스 Grux의 메테오와 공중 콤보는 서버/클라이언트가 서로 다른 상태를 만들기 쉬운 흐름이었습니다.

- AGruxMeteor를 복제 Actor로 전환 · 이동 복제 ON
- Target · Damage 등 메테오 상태를 복제 속성으로 추가
- 타이머 시작 · 투사체 활성화 · 충돌 · 데미지 · 파괴는 **서버 권위에서만** 실행
- 바닥 폭발과 캐릭터 직접 피격을 분리
- 폭발 이펙트는 서버 Multicast 전파, **Actor 삭제는 짧게 지연해** 이펙트 RPC 유실 방지
- Grux 공중 콤보 함수에 서버 권위 가드 · 널 가드 추가

CODE PATH

Actor/Projectile/GruxMeteor.{h,cpp} · Actor/Character/AI/Grux.cpp · Actor/Item/GruxItem.cpp

Core · Death / Respawn & Responsibility Split

서버 상태 복구와 클라이언트 표현 복구를 분리한, 프로젝트 전반의 단일 패턴입니다.

7-6 사망 / 부활 흐름 보정

사망 완료가 Anim Notify에만 의존하면 Notify 누락 시 부활 타이머가 시작되지 않을 수 있었고, **서버 권위 상태 복구와 클라이언트 표현 복구가 한 함수에 섞여** 있었습니다.

- 사망 완료 처리는 **서버에서 한 번만 예약**
- 사망 Multicast는 visibility · 사망 이펙트 같은 **표현만** 처리
- 3초 후 서버 타이머가 **부활 Multicast** 호출
- HP 회복 · State->SetIdleMode()는 **authority 인스턴스에서만**
- 입력 · visibility · 몽타주 정리 · 이동 복구는 각 클라이언트 표현에 맞게
- GameMode 리스폰 타이머는 **함수 로컬 핸들**로 사용해 다중 플레이어 타이머 덮어쓰기 방지

책임 분리 — 서버 권위 vs 클라이언트 표현

이 프로젝트가 반복적으로 적용한 단일 패턴

SERVER · 권위 (AUTHORITY)

- 액션 검증 · 확정
- 데미지 · 사망 판정
- HP / State 복구
- 보스 스킬 타이머·충돌·파괴
- 부활 예약

CLIENT · 표현 (PRESENTATION)

- 예측 재생 (소유 클라이언트)
- 복제 태그 → 몽타주 재생
- 피격 / 폭발 이펙트
- visibility · 입력 복구
- 몽타주 정리

연결고리는 복제와 Multicast — 서버가 상태를 확정하고, 각 클라이언트는 복제된 상태/이벤트로 같은 결과를 표현합니다.

Troubleshooting · 1-2

입력 지연과 장착/액션 데이터 불일치 — 문제 → 원인 → 해결 → 결과.

1 소유 클라이언트 평타 입력 지연

문제	기존 평타는 Server RPC → Reliable Multicast 이후에 몽타주가 재생되어, 지연 환경에서 소유 클라이언트 입력 반응이 늦어질 수 있었다.
원인	소유 클라이언트도 서버 응답을 기다린 뒤 같은 Multicast 경로로 표현을 받는 구조였다.
해결	TryAction()에서 소유 클라이언트가 조건을 통과하면 즉시 로컬 예측 재생, 서버에는 Server_TryAction(). 서버 확정 액션은 ActiveAction으로 복제하되 owner엔 COND_SkipOwner 적용.
결과	평타 경로에서 입력 즉시 반응과 서버 권위 액션 확정을 동시에 두는 기준선이 생겼다.

2 장착 아이템 / 현재 액션 데이터 불일치

문제	Anim Notify·콤보 로직이 현재 아이템·액션 데이터에 의존하지만, 서버와 클라이언트가 같은 데이터를 본다는 보장이 부족했다.
원인	장착 아이템 참조와 CurrentData 전체가 네트워크 기준으로 안정화되어 있지 않았다.
해결	선택 아이템 Actor를 서버 스폰/복제 구조로 정리하고, 현재 액션은 FActionData 전체 대신 CurrentActionTag를 복제해 클라이언트가 DataTable에서 같은 데이터를 재구성하게 했다.
결과	액션 의미를 태그로 맞추는 최소 복제 구조가 생겼고, 이후 콤보 인덱스·섹션·시작 시간 확장 기반이 마련됐다.

Troubleshooting · 3-4

보스 메테오 동기화와 사망/부활 복구 — 문제 → 원인 → 해결 → 결과.

3 보스 메테오 이펙트와 판정 불일치

문제	Grux 메테오가 클라이언트에서 안정적으로 보이지 않거나, 충돌 직후 Actor가 삭제되어 폭발 이펙트 Multicast를 놓칠 수 있었다.
원인	메테오 Actor 복제와 서버 권위 충돌 처리가 부족했고, Multicast 직후 즉시 Destroy()가 호출되는 구조였다.
해결	메테오를 복제 Actor로 전환하고 서버에서만 충돌/데미지/파괴 처리. 이펙트 Multicast 후 충돌·이동을 끄고 SetLifeSpan(1.f)로 지연 삭제.
결과	보스 스킬의 실제 판정은 서버가 결정하고, 클라이언트는 복제 Actor와 Multicast 이펙트를 관찰하는 구조로 정리됐다.

4 사망 후 부활 / 입력 복구 불안정

문제	사망 Notify 누락·타이밍 어긋남 시 부활 타이머가 시작되지 않거나, 부활 후 입력·이동 가능 상태가 복구되지 않을 수 있었다.
원인	사망 완료·부활 타이머·HP/State 복구·visibility/input 복구가 같은 흐름에 섞여 있었고 일부가 클라이언트에서도 직접 실행될 수 있었다.
해결	서버가 사망 완료와 부활 타이머를 한 번만 관리하고, 클라이언트는 Multicast로 표현 상태를 복구하도록 분리. 부활 시 Status->SetMove()로 이동 가능 상태 복원.
결과	사망/부활의 서버 권위 상태와 클라이언트 표현 책임이 더 명확해졌다.

Verification

빌드, 2인 PIE 런타임 상태, 별도 프로세스 NULL 검증으로 결과를 증빙합니다.

빌드 검증 · UnrealBuildTool

BUILD.BAT

```
Build.bat SouLikeEditor Win64 Development
-Project=...\SouLike.uproject
-WaitMutex -FromMsBuild
```

- SouLikeEditor Win64 Development 빌드 성공
- 여러 수정 단위에서 Result: Succeeded 확인
- UHT 선언 오류 · 타입 불일치 · 접근 지정자 · deprecation 경고를 빌드 로그 기준으로 수정

별도 프로세스 검증 준비

LAUNCHNULLLISTENTEST.BAT

```
Listen: /Game/_dev/Level/MainWorld?Listen
Client: 127.0.0.1 직접 접속
Option: -nosteam
```

- Steam OSS 전환 전 **NULL/직접 접속** 기준으로 전투 동기화 문제 분리
- PIE와 별도 프로세스의 Possess · BeginPlay · 복제 타이밍 차이 확인

2인 PIE 런타임 상태 확인

2

BP_Warrior0 · BP_Warrior1
스폰 확인

Idle

두 Pawn 모두
StateComponent.Type

600

두 Pawn 모두
StatusComponent.SP.Current

0

ActionComponent
ActiveAction.Sequence

- EquipComponent.SelectItem 유효 상태 확인
- PIE 로그 Error · Warning 필터 결과 **없음**

증빙 가능: 빌드 로그 · 2인 PIE 런타임 상태 · NULL 테스트 스크립트 · 날짜별 작업 로그와 코드 경로.

Key Code Paths

액션·아이템·피격·보스·사망 흐름별 핵심 코드 경로 맵입니다.

핵심 코드 경로

변경 단위가 어디에 있는지 — 포트폴리오에서 “무엇을 어디서 고쳤는지” 추적용

목적	코드 경로
통일 액션 진입점	Source/SoulLike/Component/ActionComponent.{h, cpp}
플레이어 평타 입력 연결	Source/SoulLike/Actor/Character/Player/BasePlayer.cpp
공통 캐릭터 컴포넌트 구성	Source/SoulLike/Actor/Character/BaseCharacter.cpp
장착 아이템 복제	Source/SoulLike/Component/EquipComponent.{h, cpp}
현재 액션 태그 복제	Source/SoulLike/Actor/Item/Item.{h, cpp}
피격 리액션 전파	Source/SoulLike/Actor/Character/BaseCharacter.cpp · Component/MontageComponent.cpp
보스 메테오 동기화	Source/SoulLike/Actor/Projectile/GruxMeteor.{h, cpp}
사망 / 부활 처리	Source/SoulLike/Actor/Character/Player/BasePlayer.cpp · Actor/GameMode/MainWorldGameMode.cpp
별도 프로세스 테스트	Scripts/LaunchNullListenTest.bat · Scripts/LaunchSteamMenuTest.bat

Retrospective & Summary

잘한 점, 한계, 다음 개선과 포트폴리오 요약 문장입니다.

잘한 점

- 한 번에 같아있지 않고 **평타부터** 서버 권위 액션 상태 복제 구조 적용
- 장착 아이템·액션 태그·피격·보스 메테오·사망/부활처럼 **실제 동기화 문제 지점**을 작게 나누어 수정
- 변경마다 작업 로그·검증 결과를 남겨 “**왜 그렇게 고쳤는지**”까지 설명 가능

한계

- UActionComponent는 평타 중심 1차 기준선 — 회피/스텝백/스킬/AI 전체가 완전 이전된 상태는 아님
- 콤보 인덱스·몽타주 시작 시간·섹션·재생 속도가 아직 별도 액션 상태로 정규화되지 않음
- Steam OSS 런타임 세션 검증은 준비 단계 — 2 PC/2 계정 환경 검증 필요
- Anim Notify가 여전히 일부 게임플레이 책임 보유 → 타이밍 이벤트 역할로 축소 필요

다음 개선

- 1 회피·스텝백·스킬을 TryAction() 패턴으로 이전
- 2 AI 평타/스킬도 서버 확정 액션 상태 복제로 통일
- 3 FActiveAction에 콤보 인덱스·섹션·시작 시간 등 최소 필드 추가
- 4 Net PktLag 환경에서 예측 재생·서버 보정 검증
- 5 NULL 직접 접속 통과 후 Steam OSS 세션 검증으로 진행

PORTFOLIO SUMMARY

레거시 Unreal 액션 RPG의 **전투 동기화 문제**를 해결하기 위해 UActionComponent 기반 **서버 권위 액션 상태 복제 구조**를 설계하고, GameplayTag · DataTable · Unreal Replication · RPC로 **평타 예측 재생 · 장착 아이템 복제 · 현재 액션 태그 복제 · 피격/보스 스킬/사망 부활 동기화**를 단계적으로 구현했다. 그 결과를 UnrealBuildTool 빌드 · 2인 PIE 런타임 상태 · 별도 프로세스 NULL 테스트 · 날짜별 작업 로그와 코드 경로로 증빙할 수 있다.